Network & Endpoint Security

Practical Portfolio

Section 1: Firewall & ACL Configuration

Student	vboxuser / jayy
Environment	Ubuntu VM + Cisco Packet Tracer
Date	August 2026
Status	Completed

1. Objective

Configure, test, and troubleshoot host-based and network-based access control using Linux Uncomplicated Firewall (UFW) and Cisco Access Control Lists (ACLs). Demonstrate the difference between standard and extended ACLs and verify rules with both positive and negative testing.

2. Linux Host Firewall (UFW)

2.1 Environment

- Ubuntu VM running in VirtualBox
- IP Address: **10.0.2.15** (NAT network)
- Interface: enp0s3

2.2 Configuration Steps

1. Set secure default policies:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

2. Allow essential services:

```
sudo ufw allow 22/tcp   # SSH
sudo ufw allow 80/tcp   # HTTP
sudo ufw allow 443/tcp  # HTTPS
```

3. Enable the firewall:

```
sudo ufw enable
```

2.3 Final UFW Status

```
Status: active
Default: deny (incoming), allow (outgoing)

[ 1] 22/tcp    ALLOW IN  Anywhere
[ 2] 443/tcp   ALLOW IN  Anywhere
[ 3] 80/tcp    ALLOW IN  Anywhere
(+ corresponding IPv6 rules)
```

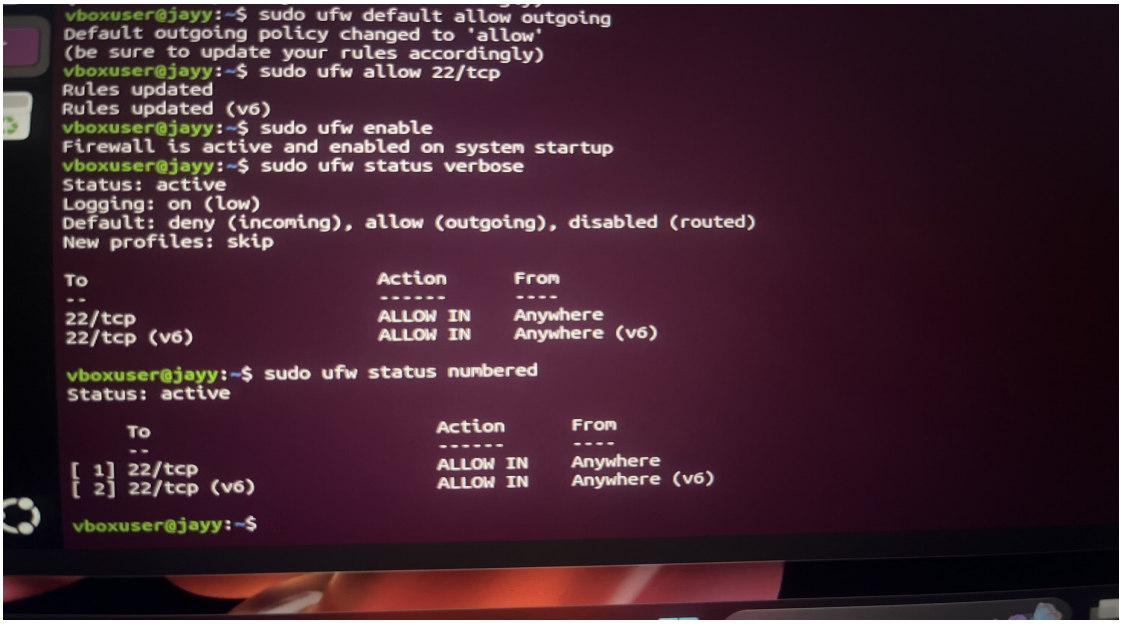

Figure 1: UFW status showing active firewall with allow rules for SSH, HTTP and HTTPS

2.4 Testing with Nmap

A Python HTTP server was started on port 80. Nmap was then used to verify the port state. Port 80 changed from **closed** to **open** once the service was listening, confirming that the UFW allow rule was effective.

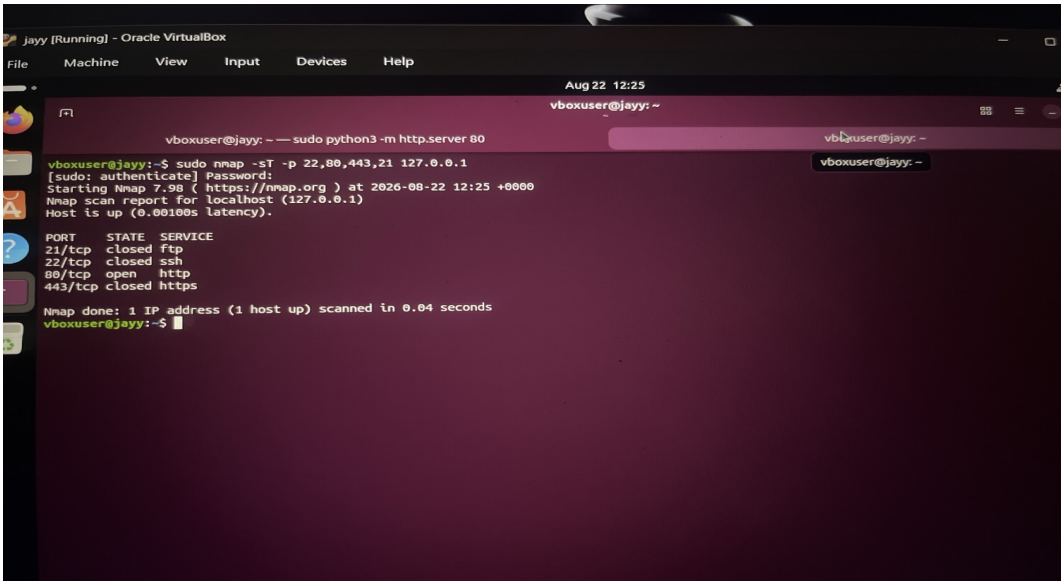

Figure 2: Nmap scan showing port 80 open after starting the Python web server

Key Observation: Traffic originating from localhost (127.0.0.1) often bypasses UFW rules. This is expected Linux behavior and was documented as an important real-world note.

3. Cisco Packet Tracer – Network ACLs

3.1 Topology

A simple two-network topology was built consisting of a Cisco 1941 router, a 2960 switch, two PCs on the LAN side, and a Server on a separate network.

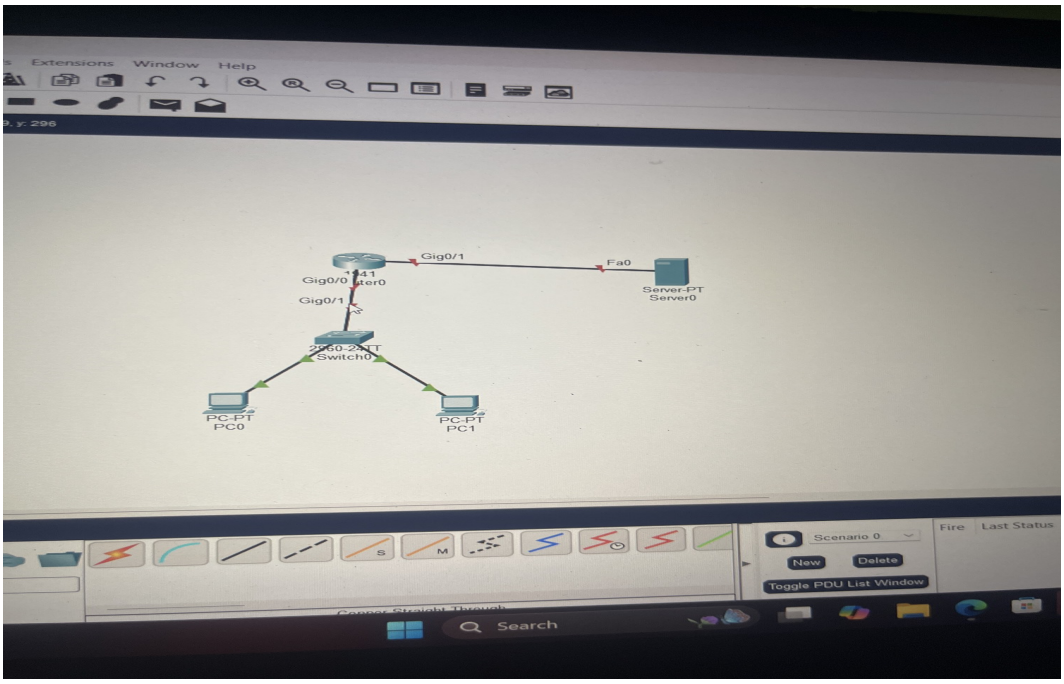

Figure 3: Packet Tracer topology – Router, Switch, PC0, PC1 and Server

3.2 Addressing Scheme

Device	Interface / IP
Router Gig0/0	192.168.1.1 /24 (LAN)
Router Gig0/1	192.168.2.1 /24 (Server network)
PC0	192.168.1.2 /24
PC1	192.168.1.3 /24
Server	192.168.2.2 /24

3.3 Standard ACL (ACL 10)

Objective: Block PC1 (192.168.1.3) from reaching the Server while still allowing PC0.

```
access-list 10 deny 192.168.1.3 0.0.0.0
access-list 10 permit any
interface GigabitEthernet0/1
ip access-group 10 out
```

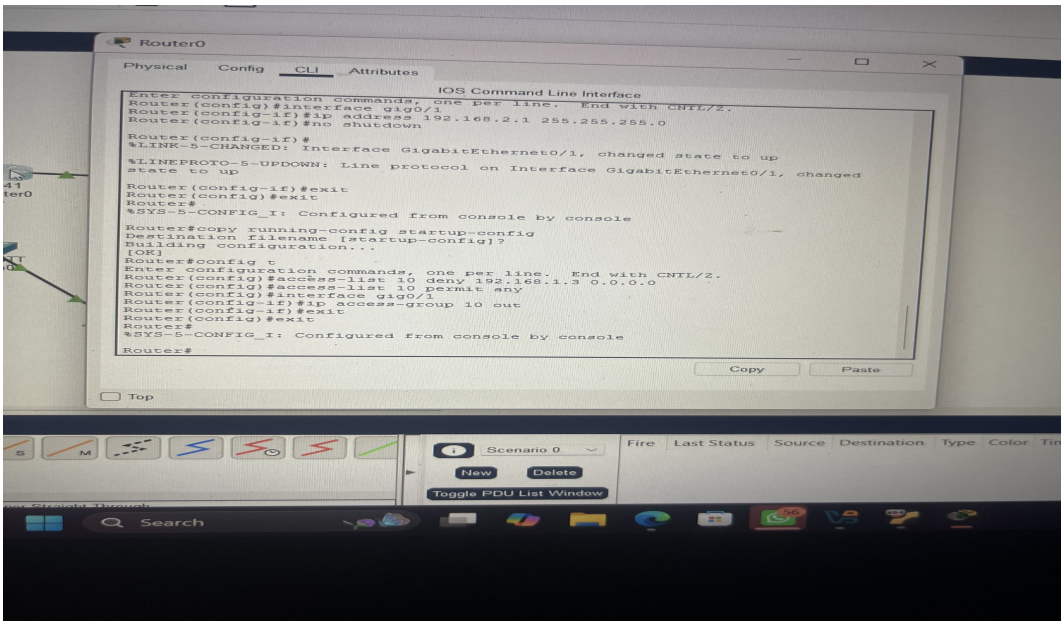

Figure 4: Standard ACL configuration on the router

Test Results – Standard ACL

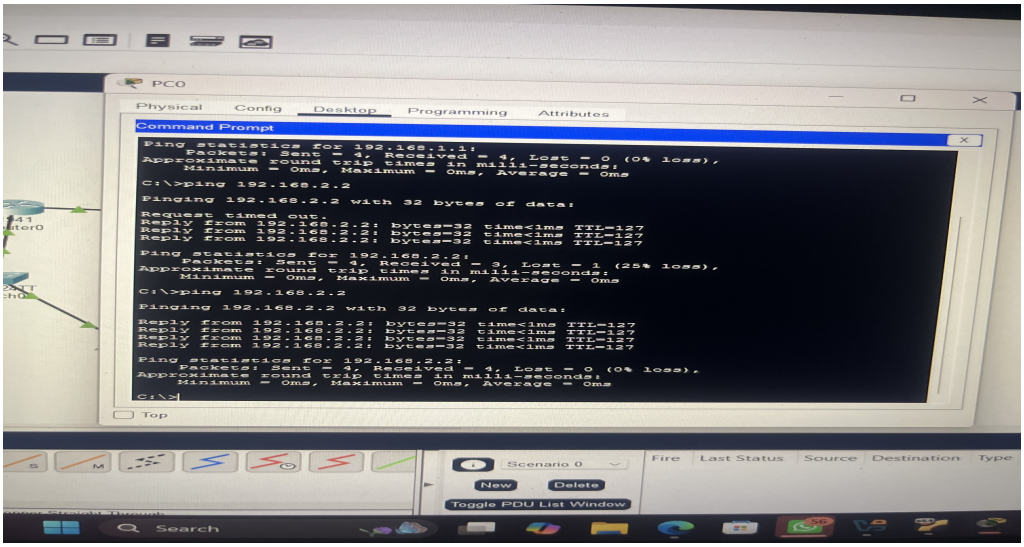

Figure 5: PC0 successfully pings the Server (allowed by ACL)

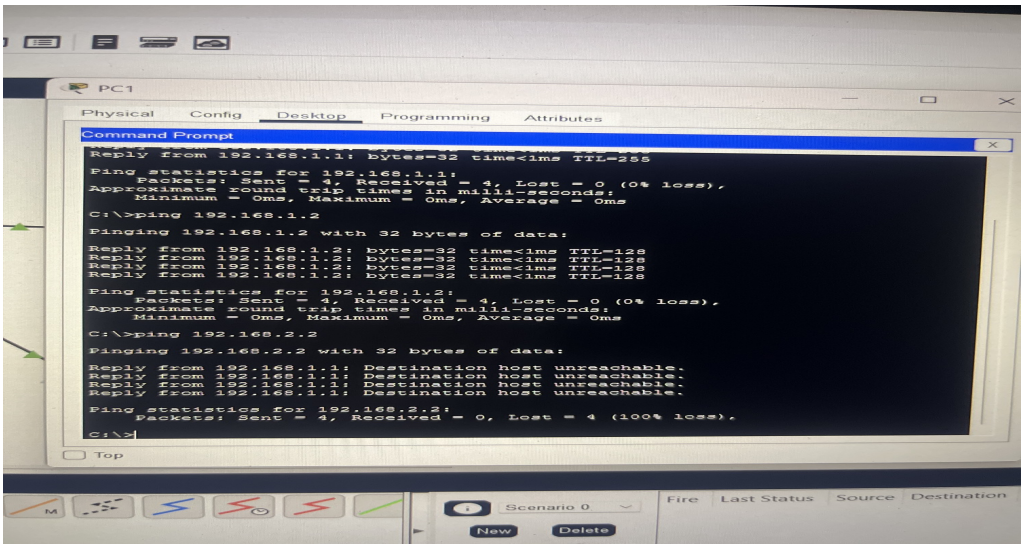

Figure 6: PC1 receives “Destination host unreachable” (blocked by ACL)

3.4 Extended ACL (ACL 100)

Objective: Allow only HTTP (TCP port 80) traffic to the Server. Deny all other traffic (including ICMP/ping) while still permitting normal traffic elsewhere in the network.

```
access-list 100 permit tcp any host 192.168.2.2 eq 80
access-list 100 deny ip any host 192.168.2.2
access-list 100 permit ip any any
interface GigabitEthernet0/1
ip access-group 100 out
```

Test Results – Extended ACL

• Ping from both PC0 and PC1 → **Failed** (Destination host unreachable)
• Web Browser (http://192.168.2.2) from both PCs → **Success** (page loaded)

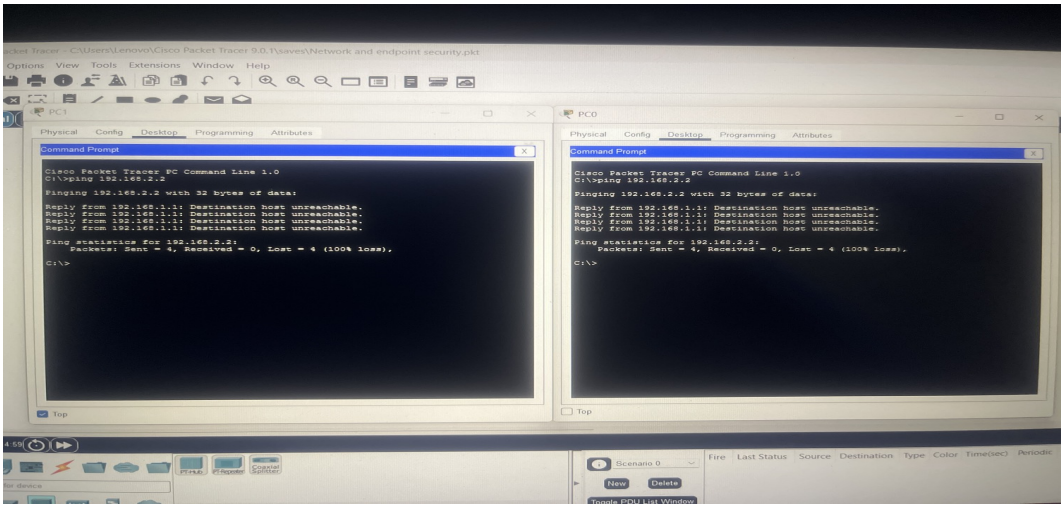

Figure 7: Both PCs blocked from pinging the Server (ICMP denied)

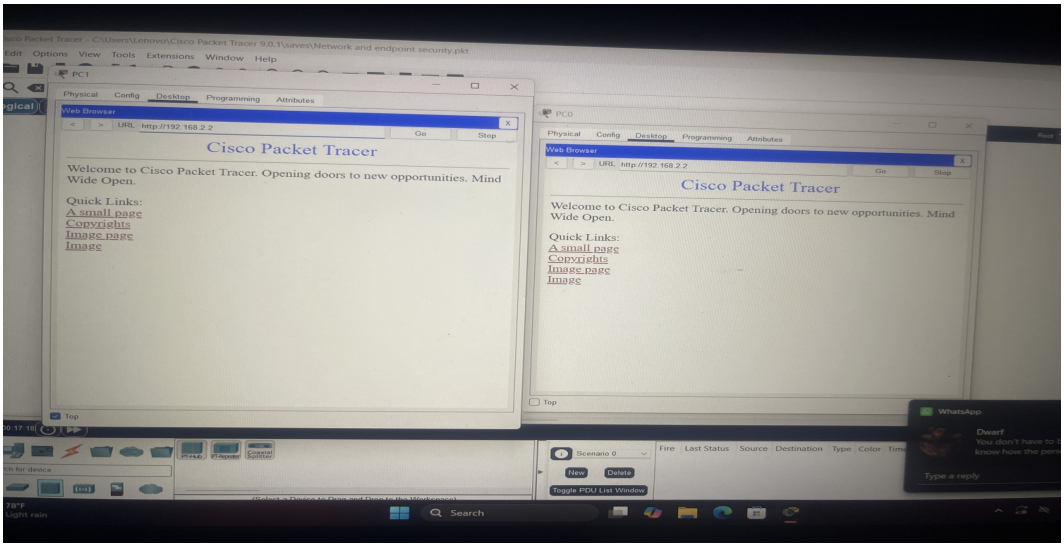

Figure 8: Both PCs successfully access the web server via HTTP (port 80 allowed)

4. Key Takeaways

- Host firewalls (UFW) protect individual endpoints using a default-deny posture.
- Network ACLs filter traffic as it crosses the router and can protect entire segments.
- Standard ACLs filter based on source IP address only.
- Extended ACLs can filter by source, destination, protocol and port number.
- Always apply ACLs in the correct direction and verify with both positive and negative tests.
- Localhost traffic frequently bypasses host firewall rules — an important real-world consideration.

5. Tools & Technologies Used

- Ubuntu Linux + UFW
- nmap, tcpdump, net-tools
- Cisco Packet Tracer (Router 1941, Switch 2960)
- Python HTTP server for service testing

Section 1 Status: Completed